Data Aware Load Balancing

Zachary Snyder, Hamed Gorjiara, and Seungmok Lee

March 15, 2017

Introduction

Here, we explore load balancing in the context of information about the data dependencies between tasks in a distributed application. Although it is possible to perform load balancing oblivious to the structure of the applications, some techniques try to use more information about the application structures to achieve better results. We focus on dynamic load balancing algorithms, as these seem to best fit the kinds of tasks that one encounters in practice at present.

Background

All of the techniques presented here are based upon work stealing. Cilk [1] showed that work stealing could achieve existentially optimal guarantees about the performance of a single program on a distributed system.

The expected execution time is bounded above by $\frac{T_1}{P} + O(T_\infty)$, where T_1 is the time it takes to serially execute the parallelizable portions of the program, P is the number of processors, and T_∞ is the amount of time it would take to run the program on an infinite number of processors (critical path time).

The space required is bounded above by $S_1 P$, where S_1 is the minimum space required to serially execute the program, and P is the number of processors.

The expected communication overhead (measured in bytes) is bounded above by $O(PT_\infty(1+n_d)S_{max})$, where P is the number of processors, T_∞ is the critical path time as above, n_d is the maximum number of times that a thread synchronizes with its parent, and S_{max} is the size of the largest activation record of any thread.

As it turns out, the parameter S_{max} ends up being the parameter of interest, as it can be quite large. Although these bounds are existentially optimal, some parameters can be tweaked, and choosing the tasks to steal more carefully than randomly may reduce some of the constants involved in the communication overhead significantly.

State of the Art

There are a few systems currently in use that take advantage of information about the application structure. We present them here to show what kind of information is being used, and what kinds of optimizations can be made with this information.

One of the major criticisms of Cilk is that it does not respect data locality [4]. Since transferring data is expensive when the data is large, migrating some tasks may be more expensive than others, and may even be more expensive than simply waiting. This is especially true when tasks are short-lived. The techniques below primarily are trying to address this concern, while building upon the work-stealing technique.

Hadoop Fair Scheduling

A buzzword of a few years ago, Hadoop [2] provides a map-reduce model of computation to application programmers. What makes Hadoop interesting in the context of structure aware scheduling is that Hadoop targets a very specific model of computation and a relatively narrow class of problems.

One of the constraints for a distributed system is fairness. Different processes from different users should all make progress are comparable rates. This can be tricky to achieve when data locality is another significant constraint, as it may be undesirable to migrate tasks, potentially compromising fairness.

In Hadoop Fair Scheduling (HFS) [5] new jobs are sorted in a list according to a hierarchical scheduling policy. Then, each job that will be assigned to the node that contains the input data. If that node is busy with other tasks, the new job can wait for a little amount of time and the scheduler can schedule other new jobs. With this algorithm, it is possible to use local data nearly 100% of the time, while maintaining relaxed a fairness guarantee.

Spark [7] is another utility that uses HFS [6], although it provides a much more general model of computation.

The reason that Hadoop and Spark get away with using HFS is that the tasks that are run are relatively small and short-lived, and the data associated with them is most always large. It is less expensive to wait than to move data. As it happens, some believe that this is likely to become more true of all distributed applications as computational needs grow.

Data Aware Work Stealing

Waiting on large data is a good way of avoiding communication overhead, but in a generalized and large scale environment, not all data may be large. For such a scenario, one might want to use something called Data Aware Work Stealing (DAWS) [4].

The idea behind DAWS is that some tasks need a lot of data and shouldn't be migrated, but there are others that don't. The scheduling algorithm puts two ready queues on each node: one that is local to the machine, and one that

can be stolen from. When a task needs to be enqueued, it is enqueued on the appropriate queue on a node that has the task's required data.

This strategy was implemented on MATRIX [3], which is another generalized task execution framework, this time specifically targeting machines of epic proportions. They also make the assumption that tasks are relatively short.

Potential Future Work

It seems to us that there is an issue with the model used by the authors of Cilk [1]. The communication overhead is tallied separately from the running time, but the reality is that the communication overhead fairly directly contributes to the running time of any given application. It would seem that a better model of cost could be developed where the communications are regarded as additional tasks in the application (data transfer has a time cost). These tasks would be contributed, in part, by the load balancer itself. Such a model should more directly model the concerns that the recent work has with data locality and the expense of migration. It may be that a truly optimal load balancer can rearrange the cost between the communication overhead and the actual task scheduling to achieve a better analytical bound (if only by a large constant) than is achieved with a data oblivious work stealing algorithm, as expressed in the new model.

References

[1] Robert D. Blumofe and Charles E. Leiserson. "Scheduling Multithreaded Computations by Work Stealing". In: *Journal of the ACM* 46.5 (Sept. 1999), pp. 720–748.

[2] Jeffry Dean and Sanjay Ghemawat. "MapReduce: Simplified Data Processing on Large Clusters". In: *Communications of the ACM* 51.1 (Jan. 2008), pp. 107–113.

[3] Anupam Rajendram, Ke Wang, and Ioan Raicu. *MATRIX: MAny-Task computing execution fabRIc as eXascale*. Illinois Institute of Technology, 2013. URL: `http://datasys.cs.iit.edu/reports/2013_GCASR13_paper_MATRIX.pdf` (visited on 03/14/2017).

[4] Ke Wang et al. "Load-balanced and Locality-aware Scheduling for Data-intensive Workloads at Extreme Scales". In: *Concurrency and Computation: Practice and Experience* 28.1 (Jan. 2016), pp. 70–94.

[5] Matei Zaharia et al. "Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling". In: *Proceedings of the 5th European conference on Computer systems*. ACM.

[6] Matei Zaharia et al. *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*. Tech. rep. University of California, Berkley, 2011. URL: `https://www2.eecs.berkeley.edu/Pubs/TechRpts/2011/EECS-2011-82.pdf` (visited on 02/14/2017).

[7] Matei Zaharia et al. "Spark: Cluster Computing with Working Sets". In: *Proceedings of the 2Nd USENIX Conference on Hot Topics in Cloud Computing*. USENIX Association, 2010, p. 10.